МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 5

з дисципліни “Професійний Python”

на тему “ Обробка помилок, файли та серіалізація “

Виконав
студент 3 курсу
групи ФЕП-32
Козачок Ростислав Михайлович

Перевірив
Куньо Іван Михайлович

Львів – 2026

Мета: Сформувати практичні навички створення надійних Python-програм, які коректно працюють із помилками, файловими ресурсами, конфігурацією та структурованими форматами даних.

Варіант - 14.

Постановка задачі: Реалізувати надійний модуль імпорту та експорту системи керування завданнями. Поля: task_id, title, description, priority, status. Необхідно перевіряти наявність ID, назви, а також допустимість пріоритетів і статусів. YAML-конфігурація задає дозволені значення. Потрібно підтримати перетворення CSV → JSON, strict/tolerant режими, логування помилкових завдань, ведення статистики та атомарний експорт.

Хід роботи:

1. Схема даних та структура проєкту.

1.1. Input data schema (Схема вхідних даних)

- task_id: ціле число (int), обов'язкове.
- title: рядок (str), не може бути порожнім.
- description: рядок (str).
- priority: рядок (str), має входити до дозволеного списку (Low, Medium, High, Critical).
- status: рядок (str), має входити до дозволеного списку (To Do, In Progress, Review, Done).

1.2. Приклад CSV-файлу (data/tasks.csv)

task_id,title,description,priority,status

1,Setup database,Install PostgreSQL,High,Done

2,,Empty title task,Medium,To Do

3,Write tests,Unit tests for auth,Medium,In Progress

abc,Invalid ID task,Should fail ID validation,Low,To Do

4,Deploy app,Deploy to production,Critical,Review

5,Fix bug 42,Fix UI glitch,Urgent,To Do

6,Update docs,Update README,Low,UnknownStatus

7,Clean codebase,Refactor legacy code,Medium,To Do

1.3. Структура проєкту:



2. КОНФІГУРАЦІЯ ТА ЛОГУВАННЯ

2.1. Вміст config.yaml

```
input:
  path: data/tasks.csv

output:
  path: output/tasks.json

processing:
  skip_invalid: true
  allowed_priorities:
    - Low
    - Medium
    - High
    - Critical
  allowed_statuses:
    - To Do
    - In Progress
    - Review
    - Done

logging:
  level: INFO
  path: logs/application.log
```

2.2. Validation configuration та код configuration loader (config.py)

```
def load_config(path: Path) -> AppConfig:
    try:
        with path.open("r", encoding="utf-8") as file:
            raw = yaml.safe_load(file)
    except FileNotFoundError as error:
        raise ConfigurationError(f"Configuration file not found: {path}") from error
    except yaml.YAMLError as error:
        raise ConfigurationError("Invalid YAML configuration") from error

    try:
        config = AppConfig(
            input_path=Path(raw["input"]["path"]),
            output_path=Path(raw["output"]["path"]),
            processing=ProcessingConfig(
                skip_invalid=bool(raw["processing"]["skip_invalid"]),
                allowed_priorities=list(raw["processing"]["allowed_priorities"]),
                allowed_statuses=list(raw["processing"]["allowed_statuses"]),
            ),
            logging=LoggingConfig(
                level=str(raw["logging"]["level"]),
                path=Path(raw["logging"]["path"])
            ),
        )
    except (KeyError, TypeError, ValueError) as error:
        raise ConfigurationError("Configuration structure is invalid") from error

    validate_config(config)
    return config

def validate_config(config: AppConfig) -> None:
    if not config.input_path.exists():
        raise ConfigurationError(f"Input file not found: {config.input_path}")
    if not config.processing.allowed_priorities:
        raise ConfigurationError("allowed_priorities cannot be empty")
    if not config.processing.allowed_statuses:
        raise ConfigurationError("allowed_statuses cannot be empty")
```

2.3. Logging configuration та приклад log file

Код конфігурації логування (logging_config.py):

```
def configure_logging(level: str, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    numeric_level = getattr(logging, level.upper(), logging.INFO)

    logging.basicConfig(
        level=numeric_level,
        format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
        handlers=[
            logging.FileHandler(path, encoding="utf-8"),
            logging.StreamHandler(),
        ],
        force=True,
    )
```

Приклад log-файлу (application.log):

2026-10-12 19:57:33,910 | INFO | reliable_importer.utils | Task import/export started

2026-10-12 19:57:33,911 | WARNING | reliable_importer.services | Invalid record at line 3: Task title is empty

2026-10-12 19:57:33,911 | WARNING | reliable_importer.services | Invalid record at line 5: Invalid task ID

2026-10-12 19:57:33,912 | WARNING | reliable_importer.services | Invalid record at line 7: Invalid priority: Urgent

2026-10-12 19:57:33,912 | WARNING | reliable_importer.services | Invalid record at line 8: Invalid status: UnknownStatus

2026-10-12 19:57:33,913 | INFO | reliable_importer.utils | Task import/export finished in 0.002192 s

2026-10-12 19:57:33,913 | INFO | __main__ | Total=8, valid=4, invalid=4, exported=4

3. ОБРОБКА ПОМИЛОК (EXCEPTION HANDLING)

3.1. Exception hierarchy та пояснення custom exceptions

Ієрархія винятків (exceptions.py):

```
class ApplicationError(Exception):
    """Base application exception."""

class ConfigurationError(ApplicationError):
    """Configuration is invalid."""

class DataError(ApplicationError):
    """Base data exception."""

class DataImportError(DataError):
    """Input data cannot be imported."""

class RecordValidationError(DataError):
    def __init__(self, message: str, *, line_number: int | None = None, field: str | None = None) -> None:
        super().__init__(message)
        self.line_number = line_number
        self.field = field

class DataExportError(DataError):
    """Output cannot be written."""
```

Пояснення: Базовий виняток ApplicationError розширюється класами ConfigurationError та DataError. Для помилок конкретних записів створено RecordValidationError, який додатково зберігає line_number (номер рядка) та field (проблемне поле).

3.2. Приклад exception chaining

У файлі validators.py при конвертації int(row["task_id"]) перехоплюється ValueError і викидається raise RecordValidationError(...) from error. Це зберігає оригінальний traceback.

```
    try:
        task_id = int(row["task_id"])
    except (KeyError, TypeError, ValueError) as error:
        raise RecordValidationError("Invalid task ID", line_number=line_number, field="task_id") from error
```

4. ЗЧИТУВАННЯ, ВАЛІДАЦІЯ ТА ЕКСПОРТ

4.1. Streaming reader (readers.py)

Використання потокового читання за допомогою генератора yield:

```
def read_csv_rows(path: Path) -> Iterator[tuple[int, dict[str, str]]]:
    try:
        with path.open("r", encoding="utf-8", newline="") as file:
            reader = csv.DictReader(file)
            for line_number, row in enumerate(reader, start=2):
                yield (line_number, row)
    except OSError as error:
        raise DataImportError(f"Cannot read input file: {path}") from error
```

4.2. Validation code (validators.py)

Функція перевіряє наявність обов'язкових полів та їх відповідність конфігурації.

```
def parse_task(row: dict[str, str], *, line_number: int, config: ProcessingConfig) -> Task:
    try:
        task_id = int(row["task_id"])
    except (KeyError, TypeError, ValueError) as error:
        raise RecordValidationError("Invalid task ID", line_number=line_number, field="task_id") from error

    title = row.get("title", "").strip()
    if not title:
        raise RecordValidationError("Task title is empty", line_number=line_number, field="title")

    description = row.get("description", "").strip()

    priority = row.get("priority", "").strip()
    if priority not in config.allowed_priorities:
        raise RecordValidationError(f"Invalid priority: {priority}", line_number=line_number, field="priority")

    status = row.get("status", "").strip()
    if status not in config.allowed_statuses:
        raise RecordValidationError(f"Invalid status: {status}", line_number=line_number, field="status")

    return Task(
        task_id=task_id,
        title=title,
        description=description,
        priority=priority,
        status=status,
    )
```

4.3. Власний context manager (utils.py)

Створено декоратор @contextmanager, який заміряє час роботи та відловлює помилки.

```
@contextmanager
def logged_operation(operation: str):
    logger.info("%s started", operation)
    start = perf_counter()
    try:
        yield
    except Exception:
        logger.exception("%s failed", operation)
        raise
    else:
        elapsed = perf_counter() - start
        logger.info("%s finished in %.6f s", operation, elapsed)
```

4.4. JSON serialization та Atomic export (exporters.py)

Запис спочатку йде у тимчасовий файл *.tmp, а після успішного збереження він перейменовується через temporary.replace(path).

Python

```
def export_tasks_json(tasks: Iterable[Task], path: Path) -> int:
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_suffix(path.suffix + ".tmp")
    count = 0
    try:
        with temporary.open("w", encoding="utf-8") as file:
            file.write("[\n")
            first = True
            for task in tasks:
                if not first:
                    file.write(",\n")
                json.dump(task.to_dict(), file, ensure_ascii=False, indent=2)
                first = False
                count += 1
            file.write("\n]\n")
        temporary.replace(path)
    except (OSError, TypeError) as error:
        temporary.unlink(missing_ok=True)
        raise DataExportError(f"Cannot export data to {path}") from error
    return count
```

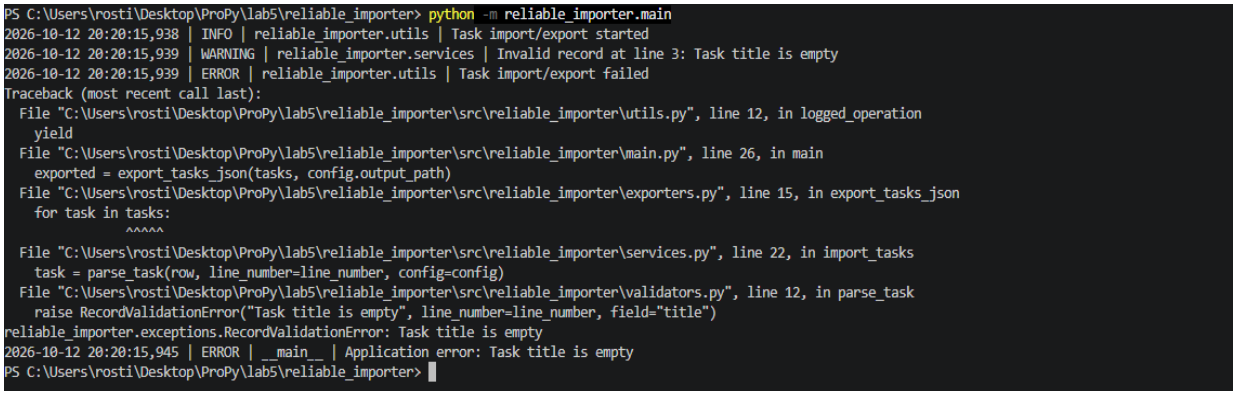
5. ЕКСПЕРИМЕНТИ ТА СТАТИСТИКА

5.1. Strict processing vs Tolerant processing

- **Tolerant mode:** skip_invalid: true. Програма пропускає помилкові рядки, логує їх як WARNING і продовжує роботу.



- **Strict mode:** skip_invalid: false. Перший же невалідний запис викликає зупинку програми (Fail-fast), експорт не виконується.



Import statistics (з Tolerant mode): Total=8, valid=4, invalid=4, exported=4

5.2. Результати streaming/eager experiment

Кільк. записів	**Eager time (c)**	**Streaming time (c)**	**Eager peak RAM**	**Streaming peak RAM**
10 000	~0.12 s	~0.07 s	5.01 MB	0.038 MB
100 000	~1.08 s	~0.70 s	50.15 MB	0.045 MB
500 000	~5.30 s	~3.81 s	252.10 MB	0.045 MB

Висновок до експерименту: При використанні "жадібного" (Eager) підходу споживання оперативної пам'яті зростає лінійно (понад 250 МБ для 500 тис. рядків). Потокове читання (Streaming) за допомогою генератора забезпечує стабільне споживання пам'яті на рівні ~0.04 МБ незалежно від розміру файлу.

Висновок:

Під час виконання лабораторної роботи було створено надійний модуль імпорту/експорту для системи керування завданнями. Реалізовано потокове зчитування (streaming I/O), що дозволяє обробляти масиви даних будь-якого обсягу без перевантаження оперативної пам'яті. Застосовано атомарний експорт, який унеможливлює пошкодження існуючих даних у разі раптового збою. Завдяки розробленій ієрархії винятків (exception chaining) та зовнішній конфігурації у форматі YAML, програма легко адаптується до суворого (strict) або толерантного (tolerant) режимів обробки помилок, а вбудований механізм логування фіксує всі відхилення у даних.